In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
tracks_df = pd.read_csv("tracks_1.csv")
artists_df = pd.read_csv("artists_1.csv")
```

In [3]:
```
tracks_df.head()
artists_df.head()
```

Out[3]:

	id	followers	genres	name	popularity
0	0DheY5irMjBUeLybbCUEZ2	0.0	[]	Armid & Amir Zare Pashai feat. Sara Rouzbehani	0
1	0DlhY15l3wsrnlfGio2bjU	5.0	[]	ปูนา ภาวิณี	0
2	0DmRESX2JknGPQyO15yxg7	0.0	[]	Sadaa	0
3	0DmhnbHjm1qw6NCYPeZNgJ	0.0	[]	Tra'gruda	0
4	0Dn11fWM7vHQ3rinvWEl4E	2.0	[]	Ioannis Panoutsopoulos	0

In [4]:
```
tracks_df.shape
artists_df.shape
```

Out[4]:
```
(1104349, 5)
```

In [5]:
```
tracks_df.columns
artists_df.columns
```

Out[5]:
```
Index(['id', 'followers', 'genres', 'name', 'popularity'], dtype='object')
```

In [6]:
```
artists_df.columns
```

Out[6]:
```
Index(['id', 'followers', 'genres', 'name', 'popularity'], dtype='object')
```

In [7]:
```
tracks_df.columns
```

Out[7]:
```
Index(['id', 'name', 'popularity', 'duration_ms', 'explicit', 'artists',
       'id_artists', 'release_date', 'danceability', 'energy', 'key',
       'loudness', 'mode', 'speechiness', 'acousticness', 'instrumentalness',
       'liveness', 'valence', 'tempo', 'time_signature'],
      dtype='object')
```

In [8]:
```
tracks_df.isnull().sum()
```

Out[8]:
```
id                   0
name                71
```

```
popularity           0
duration_ms          0
explicit             0
artists              0
id_artists           0
release_date         0
danceability         0
energy               0
key                  0
loudness             0
mode                 0
speechiness          0
acousticness         0
instrumentalness     0
liveness             0
valence              0
tempo                0
time_signature       0
dtype: int64
```

In [9]:

```
artists_df.isnull().sum()
```

Out[9]:

```
id             0
followers     13
genres         0
name           0
popularity     0
dtype: int64
```

In [10]:

```
tracks_df.describe()
```

Out[10]:

	popularity	duration_ms	explicit	danceability	energy	key	586
count	586672.000000	5.866720e+05	586672.000000	586672.000000	586672.000000	586672.000000	
mean	27.570053	2.300512e+05	0.044086	0.563594	0.542036	5.221603	
std	18.370642	1.265261e+05	0.205286	0.166103	0.251923	3.519423	
min	0.000000	3.344000e+03	0.000000	0.000000	0.000000	0.000000	
25%	13.000000	1.750930e+05	0.000000	0.453000	0.343000	2.000000	
50%	27.000000	2.148930e+05	0.000000	0.577000	0.549000	5.000000	
75%	41.000000	2.638670e+05	0.000000	0.686000	0.748000	8.000000	
max	100.000000	5.621218e+06	1.000000	0.991000	1.000000	11.000000	

In [11]:

```
artists_df.describe()
```

Out[11]:

	followers	popularity
count	1.104336e+06	1.104349e+06
mean	1.074304e+04	9.083884e+00
std	2.609554e+05	1.376310e+01
min	0.000000e+00	0.000000e+00

	followers	popularity
25%	1.000000e+01	0.000000e+00
50%	5.900000e+01	2.000000e+00
75%	4.510000e+02	1.400000e+01
max	7.890023e+07	1.000000e+02

In [12]:
```
tracks_df.describe(include='object')
```

Out[12]:

	id	name	artists	id_artists	release_date
count	586672	586601	586672	586672	586672
unique	586672	446474	114030	115062	19700
top	35iwgR4jXetI318WEWsa1Q	Summertime	['Die drei ???']	['3meJIgRw7YleJrmbpbJK6S']	1998-01-01
freq	1	101	3856	3856	2893

In [13]:
```
tracks_df.dtypes
```

Out[13]:
```
id                  object
name                object
popularity           int64
duration_ms          int64
explicit             int64
artists             object
id_artists          object
release_date        object
danceability       float64
energy             float64
key                  int64
loudness           float64
mode                 int64
speechiness        float64
acousticness       float64
instrumentalness   float64
liveness           float64
valence            float64
tempo              float64
time_signature       int64
dtype: object
```

In [14]:
```
tracks_df['popularity'] = tracks_df['popularity'].
```

```
  File "/tmp/ipykernel_4019/272585800.py", line 1
    tracks_df['popularity'] = tracks_df['popularity'].
                                                      ^
SyntaxError: invalid syntax
```

In [15]:
```
artists_df['popularity'] = artists_df['popularity'].astype(int)
```

In [16]:
```
tracks_df['popularity'] = tracks_df['popularity'].astype(int)
```

In [17]:
```
artists_df['genres'] = artists_df['genres'].astype('category')
```

In [18]:
```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

artists_df[['popularity', 'followers']] = scaler.fit_transform(
    artists_df[['popularity', 'followers']]
)

tracks_df[['popularity', 'danceability', 'energy', 'tempo', 'duration_ms']] = sc
    tracks_df[['popularity', 'danceability', 'energy', 'tempo', 'duration_ms']]
)

artists_df.head()
tracks_df.head()
```

Out[18]:

	id	name	popularity	duration_ms	explicit	artists	
0	35iwgR4jXetI318WEWsa1Q	Carve	0.06	0.021994	0	['Uli']	['45tIt06X
1	021ht4sdgPcrDgSk7JTbKY	Capítulo 2.16 - Banquero Anarquista	0.00	0.016885	0	['Fernando Pessoa']	['14jtPCOoNZ
2	07A5yehtSnoedViJAZkNnc	Vivo para Quererte - Remasterizado	0.00	0.031737	0	['Ignacio Corsini']	['5LiOoJbxVSA
3	08FmqUhxtyLTn6pAh6bk45	El Prisionero - Remasterizado	0.00	0.030895	0	['Ignacio Corsini']	['5LiOoJbxVSA
4	08y9GfoqCWfOGsKdwojr5e	Lady of the Evening	0.00	0.028434	0	['Dick Haymes']	['3BiJGZsyX9

In [19]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

artists_df['genres_encoded'] = le.fit_transform(artists_df['genres'])
```

In [20]:
```
artists_encoded_df = pd.get_dummies(artists_df, columns=['genres'])
artists_encoded_df.head()
```

```
---------------------------------------------------------------------------
MemoryError                               Traceback (most recent call last)
/tmp/ipykernel_4019/2398488716.py in <module>
----> 1 artists_encoded_df = pd.get_dummies(artists_df, columns=['genres'])
      2 artists_encoded_df.head()

~/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/reshape.py in get_du
mmies(data, prefix, prefix_sep, dummy_na, columns, sparse, drop_first, dtype)
    934         for (col, pre, sep) in zip(data_to_encode.items(), prefix, prefi
x_sep):
```

```
    935             # col is (column_name, column), use just column data here
--> 936             dummy = _get_dummies_1d(
    937                 col[1],
    938                 prefix=pre,

~/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/reshape.py in _get_d
ummies_1d(data, prefix, prefix_sep, dummy_na, sparse, drop_first, dtype)
   1057     else:
   1058         # take on axis=1 + transpose to ensure ndarray layout is column-
major
-> 1059         dummy_mat = np.eye(number_of_cols, dtype=dtype).take(codes, axis
=1).T
   1060
   1061         if not dummy_na:

MemoryError: Unable to allocate 50.2 GiB for an array with shape (48788, 110434
9) and data type uint8
```

In [21]:

```
top_genres = artists_df['genres'].value_counts().head(10).index
artists_small = artists_df[artists_df['genres'].isin(top_genres)]
```

In [22]:

```
artists_encoded_df.head()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipykernel_4019/2504396676.py in <module>
----> 1 artists_encoded_df.head()

NameError: name 'artists_encoded_df' is not defined
```

In [23]:

```
artists_encoded_df = pd.get_dummies(artists_small, columns=['genres'])
artists_encoded_df.head()
```

```
---------------------------------------------------------------------------
MemoryError                               Traceback (most recent call last)
/tmp/ipykernel_4019/1907491007.py in <module>
----> 1 artists_encoded_df = pd.get_dummies(artists_small, columns=['genres'])
      2 artists_encoded_df.head()

~/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/reshape.py in get_du
mmies(data, prefix, prefix_sep, dummy_na, columns, sparse, drop_first, dtype)
    934         for (col, pre, sep) in zip(data_to_encode.items(), prefix, prefi
x_sep):
    935             # col is (column_name, column), use just column data here
--> 936             dummy = _get_dummies_1d(
    937                 col[1],
    938                 prefix=pre,

~/anaconda3/lib/python3.9/site-packages/pandas/core/reshape/reshape.py in _get_d
ummies_1d(data, prefix, prefix_sep, dummy_na, sparse, drop_first, dtype)
   1057     else:
   1058         # take on axis=1 + transpose to ensure ndarray layout is column-
major
-> 1059         dummy_mat = np.eye(number_of_cols, dtype=dtype).take(codes, axis
=1).T
   1060
   1061         if not dummy_na:
```

```
MemoryError: Unable to allocate 36.7 GiB for an array with shape (48788, 807750)
and data type uint8
```

In [24]:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
artists_df['genres_encoded'] = le.fit_transform(
    artists_df['genres'].astype(str)
)

artists_df[['genres', 'genres_encoded']].head()
```

Out[24]:

	genres	genres_encoded
0	[]	48787
1	[]	48787
2	[]	48787
3	[]	48787
4	[]	48787

In [5]:

```
tracks_df['explicit'].unique()
```

Out[5]:

```
array([0, 1])
```

In [6]:

```
tracks_df['explicit'].dtype
```

Out[6]:

```
dtype('int64')
```

In [7]:

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
tracks_df['explicit_encoded'] = le.fit_transform(tracks_df['explicit'])
```

In [8]:

```
tracks_df[['explicit', 'explicit_encoded']].head()
```

Out[8]:

	explicit	explicit_encoded
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

In [9]:

```
tracks_df.describe()
```

Out[9]:

	popularity	duration_ms	explicit	danceability	energy	key	586
count	586672.000000	5.866720e+05	586672.000000	586672.000000	586672.000000	586672.000000	
mean	27.570053	2.300512e+05	0.044086	0.563594	0.542036	5.221603	
std	18.370642	1.265261e+05	0.205286	0.166103	0.251923	3.519423	
min	0.000000	3.344000e+03	0.000000	0.000000	0.000000	0.000000	
25%	13.000000	1.750930e+05	0.000000	0.453000	0.343000	2.000000	
50%	27.000000	2.148930e+05	0.000000	0.577000	0.549000	5.000000	
75%	41.000000	2.638670e+05	0.000000	0.686000	0.748000	8.000000	
max	100.000000	5.621218e+06	1.000000	0.991000	1.000000	11.000000	

In [10]:

```
from sklearn.preprocessing import StandardScaler
```

In [11]:

```
artists_join_df = artists_df.set_index('id')num_cols = ['popularity', 'danceabil
scaler = StandardScaler()

tracks_df[num_cols] = scaler.fit_transform(tracks_df[num_cols])
```

In [12]:

```
tracks_df[num_cols].describe()
```

Out[12]:

	popularity	danceability	energy	tempo	duration_ms
count	5.866720e+05	5.866720e+05	5.866720e+05	5.866720e+05	5.866720e+05
mean	-1.923330e-14	-8.203950e-17	-2.363092e-16	7.165120e-16	-6.469944e-16
std	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00	1.000001e+00
min	-1.500768e+00	-3.393048e+00	-2.151596e+00	-3.980128e+00	-1.791784e+00
25%	-7.931168e-01	-6.658166e-01	-7.900676e-01	-7.682030e-01	-4.343627e-01
50%	-3.103067e-02	8.071029e-02	2.764343e-02	-3.631413e-02	-1.198028e-01
75%	7.310555e-01	7.369315e-01	8.175682e-01	5.999225e-01	2.672639e-01
max	3.942704e+00	2.573147e+00	1.817875e+00	4.297668e+00	4.260917e+01

In [3]:

```
tracks_df[['id', 'artists']].head()
artists_df[['id', 'name']].head()
```

Out[3]:

	id	name
0	0DheY5irMjBUeLybbCUEZ2	Armid & Amir Zare Pashai feat. Sara Rouzbehani
1	0DlhY15l3wsrnlfGio2bjU	ปูนา ภาวิณี
2	0DmRESX2JknGPQyO15yxg7	Sadaa
3	0DmhnbHjm1qw6NCYPeZNgJ	Tra'gruda
4	0Dn11fWM7vHQ3rinvWEl4E	Ioannis Panoutsopoulos

In [4]:
```
tracks_df['artist_id'] = tracks_df['artists'].apply(
    lambda x: x.strip("[]").replace("'", "").split(",")[0]
)
```

In [5]:
```
merged_df = pd.merge(
    tracks_df,
    artists_df,
    left_on='artist_id',
    right_on='id',
    how='inner'
)
```

In [6]:
```
merged_df.head()
```

Out[6]:

id_x	name_x	popularity_x	duration_ms	explicit	artists	id_artists	release_date	danceability	ener

0 rows × 26 columns

In [7]:
```
merged_df = pd.merge(
    tracks_df,
    artists_df,
    left_on='artist_id',
    right_on='id',
    how='left'
)
```

In [8]:
```
merged_df.shape
```

Out[8]:
```
(586672, 26)
```

In [9]:
```
tracks_df['artist_id'] = (
    tracks_df['artists']
    .astype(str)
    .str.strip("[]")
    .str.replace("'", "")
    .str.split(",")
    .str[0]
    .str.strip()
)
```

In [10]:
```
artists_join_df = artists_df.set_index('id')
```

In [11]:
```
joined_df = tracks_df.join(
    artists_join_df,
    on='artist_id',
    how='left',
```

```
    rsuffix='_artist'
)
```

In [12]:
```
joined_df.head()
```

Out[12]:

	id	name	popularity	duration_ms	explicit	artists	
0	35iwgR4jXetI318WEWsa1Q	Carve	6	126903	0	['Uli']	['45tIt06X
1	021ht4sdgPcrDgSk7JTbKY	Capítulo 2.16 - Banquero Anarquista	0	98200	0	['Fernando Pessoa']	['14jtPCOoNZ
2	07A5yehtSnoedViJAZkNnc	Vivo para Quererte - Remasterizado	0	181640	0	['Ignacio Corsini']	['5LiOoJbxVSA
3	08FmqUhxtyLTn6pAh6bk45	El Prisionero - Remasterizado	0	176907	0	['Ignacio Corsini']	['5LiOoJbxVSA
4	08y9GfoqCWfOGsKdwojr5e	Lady of the Evening	0	163080	0	['Dick Haymes']	['3BiJGZsyX9

5 rows × 25 columns

In [13]:
```
joined_df.shape
```

Out[13]:
```
(586672, 25)
```

In [14]:
```
left_join_df = tracks_df.join(
    artists_join_df,
    on='artist_id',
    how='left',
    rsuffix='_artist'
)
```

In [15]:
```
left_join_df.shape
```

Out[15]:
```
(586672, 25)
```

In [16]:
```
right_join_df = tracks_df.join(
    artists_join_df,
    on='artist_id',
    how='right',
    rsuffix='_artist'
)
```

In [17]:
```
right_join_df.shape
```

Out[17]:
```
(1104349, 25)
```

In [18]:
```
inner_join_df = tracks_df.join(
    artists_join_df,
    on='artist_id',
    how='inner',
    rsuffix='_artist'
)
```

In [19]:
```
inner_join_df.shape
```

Out[19]:
```
(0, 25)
```

In [20]:
```
outer_join_df = tracks_df.join(
    artists_join_df,
    on='artist_id',
    how='outer',
    rsuffix='_artist'
)
```

In [21]:
```
outer_join_df.shape
```

Out[21]:
```
(1691021, 25)
```

In [22]:
```
thankyou!!!
```

```
  File "/tmp/ipykernel_4853/3524816938.py", line 1
    thankyou!!!
            ^
SyntaxError: invalid syntax
```

In [ ]: